



MailDay Matching

CONFIDENTIAL

Security & Quality Audit Report

A plain-English review of the MailDay admin platform,
the risks we found, and what to fix first.

PREPARED FOR

MailDay Leadership

DATE

May 19, 2026

VERSION

1.0

Table of Contents

1. Executive Summary	3
2. What's Working Well	5
3. Critical Issues — Fix This Week	6
3.1 Login key stored where any guest script can grab it	6
3.2 Outside delivery doors will open without checking ID	7
3.3 Anyone can change a customer's shipping address	8
3.4 Receipt upload accepts any file from anyone	9
3.5 Influencer commissions can be inflated by anyone	10
4. Important Issues — Fix This Month	11
5. Useful Improvements — Next Quarter	15
6. The "At Risk" Display Bug	16
7. Recommended Roadmap	17
8. Glossary of Terms	19

1. Executive Summary

MailDay Matching is the internal admin web app your team uses every day to manage pen-pal matching, members, donations, influencer partnerships, and monthly pack delivery. It connects to Shopify, ReCharge, Klaviyo, Slack, and Anthropic (the AI that suggests matches), and stores all of its information in a database called Supabase.

Bottom line: the app is well-built and the team has clearly invested in structure and clarity. It is also a one-person admin tool that has quietly grown into a system holding real customer and children's information — and several of the doors and locks have not kept pace with that growth. None of the issues below are catastrophic on their own, but a few of them should be fixed within the next week, and a few more before the business doubles in size.

How serious is what we found?



What does each level mean?

- **CRITICAL** A real attacker (or a curious customer) could realistically cause damage today. **Fix this week.**
- **IMPORTANT** A weakness that is unlikely to be exploited tomorrow, but will hurt as you grow. **Fix this month.**
- **IMPROVEMENT** A clear best-practice gap. Not urgent; bake into the next sprint or two.
- **NICE-TO-HAVE** Polish, future-proofing, or convenience. Plan for the next quarter.

The three things we recommend fixing first

DO THIS WEEK · #1 **Make webhook secrets mandatory.** Right now, if the "secret password" for our Shopify, ReCharge, or Klaviyo connections is not configured, the server still accepts incoming messages and acts on them. That is the online equivalent of a delivery door that opens regardless of whether the courier shows ID. A bored attacker who guesses the URL can create fake parent accounts, log fake donations, or end real matches. (See section 3.2)

DO THIS WEEK · #2 **Stop trusting the email field on public forms.** Three public pages — Enroll, Give-a-Key PO-Box, and the affiliate tracker — accept an email address and then change records associated with that email, with no proof the person sending the form actually owns it. The most direct consequence: anyone who knows a customer's email can change their mailing address. (See sections 3.3 and 3.5)

DO THIS WEEK · #3 **Stop storing the admin login key in a place every script on the page can read.** The token that proves you are logged in is stored in the browser's "localStorage", which any piece of JavaScript that runs on the page can read. If a third-party widget were ever added — or a single cross-site scripting bug crept in — a 30-day admin login could be stolen silently. (See section 3.1)

What we did NOT find

- No evidence the database has been tampered with.
- No SQL injection vulnerabilities in the code we reviewed.
- No passwords stored in plain text (they are properly hashed).
- No obvious exposed API keys in the public-facing frontend code.

2. What's Working Well

Before the gaps, the wins. A lot of the foundation is correct, and that matters because most of the fixes below are small additions on top of a solid base — not rewrites.

✓ **DONE RIGHT** **Passwords are stored securely.** Even if someone walked out of the office with the entire database, they would not see anyone's actual password — only a one-way scramble of it (called a "bcrypt hash"). This is the industry standard and the code uses it correctly.

✓ **DONE RIGHT** **Two roles are clearly separated.** There are two kinds of users: **Admin** and **VA** (virtual assistant). The code consistently hides sensitive fields like mailing addresses, internal notes, and safety flags from the VA role. That layered approach is exactly what you want.

✓ **DONE RIGHT** **The code is well organised.** Every feature lives in its own file, the database queries are parameterised (which blocks a whole class of attacks called "SQL injection"), and the project uses industry-standard tools (TypeScript, React, Express, Supabase, Tailwind). A new developer joining the team would be productive within a week.

✓ **DONE RIGHT** **Incoming webhook signatures CAN be checked.** The code knows how to verify that a message from Shopify is really from Shopify (and same for ReCharge and Klaviyo). The mechanism exists — the only problem is that it can be disabled by forgetting to set one environment variable. See section 3.2 for how to make it mandatory.

✓ **DONE RIGHT** **Logs do not show sensitive data.** The logging tool (pino) is configured to hide login keys and session cookies from log files. If logs are ever shared with a vendor for debugging, they will not contain those secrets.

✓ **DONE RIGHT** **The user interface is consistent and accessible.** The team picked a high-quality component library (shadcn/Radix) so things like dialogs, dropdowns, and keyboard navigation work the way users expect. The mobile layout works. Icons have labels for screen readers.

3. Critical Issues — Fix This Week

These are issues where a motivated person could cause real damage today, with skills that are freely taught online. We strongly recommend addressing all five within the next week.

CRITICAL	3.1 Login key stored where any guest script can grab it	Effort: Medium
WHAT IT IS	When you sign in, the app puts your 30-day login key into a part of the browser called <i>localStorage</i> . Anything else loaded on the page — a tracking script, an ad-blocker extension, a future copy-paste of code from a tutorial — can read it.	
ANALOGY	<i>Imagine the office key is left in the top drawer of every reception desk in every coffee shop you visit, "just in case you forget yours". You probably will never have a problem. But the day someone you don't know sits at that desk, your office is open to them for the next 30 days.</i>	
WHAT AN ATTACKER COULD DO	Steal your login key once and use it for a month. They would not need your password. They could read every parent, child, and donation record, and approve or reject Give-a-Key applications.	
WHAT TO FIX	Move the login key into an HttpOnly cookie (a special kind of cookie that browser scripts cannot read), and turn on a security feature called Content Security Policy (CSP) that the code currently has switched off.	
EFFORT	Roughly 1–2 developer days. The login flow has to be moved from "send me a token" to "set me a cookie", and the CSP rules need to be tuned for the third-party fonts the app uses.	

For the dev team: auth context lives in `artifacts/mailday/src/lib/auth.tsx` ; CSP is disabled in `artifacts/api-server/src/app.ts` . Add cookie-based auth, set `SameSite=Lax; Secure; HttpOnly` , and re-enable helmet's CSP with a whitelist for Google Fonts.

CRITICAL

3.2 Outside delivery doors will open without checking ID

Effort: Small

WHAT IT IS	When Shopify, ReCharge, or Klaviyo send us a message (called a "webhook"), the code is supposed to check a secret password that proves the message is genuine. If that secret is not configured, the code currently logs a warning and <i>accepts the message anyway</i> .
ANALOGY	<i>It's a back door at the warehouse with a card reader. If the card reader is unplugged, the door is supposed to stay locked. Right now it stays open.</i>
WHAT AN ATTACKER COULD DO	If even one of the three secrets is missing, anyone on the internet who guesses the webhook URL can: create fake parent accounts, log fake donations to the Give-a-Key fund, mark real subscriptions as cancelled (which ends matches and triggers rematch emails), or fake "I opened your email" events to skew metrics.
WHAT TO FIX	Make the secret <i>required</i> — if it is missing, the webhook endpoint should refuse all messages with a 503 error and the operator should be paged. The change is a one-line condition in three files.
EFFORT	Half a day, including checking the three production secrets are set.

For the dev team: see `artifacts/api-server/src/routes/webhooks.ts` lines 47–55 (Shopify), 148–156 (ReCharge), and 393–401 (Klaviyo). The pattern `if (secret) { ...verify... } else { warn }` should become `if (!secret) return 503; ...verify...`.

CRITICAL

3.3 Anyone can change a customer's shipping address

Effort: Medium

WHAT IT IS	The public "Enroll my children" form (<code>/enroll</code>) and the Give-a-Key PO-Box form ask for an email address, look up the matching customer in the database, and then <i>overwrite</i> that customer's mailing address with whatever was typed into the form. There is no proof the person filling out the form is the actual customer.
ANALOGY	<i>It's like the front desk of an apartment building updating your forwarding address because someone in line said "I'm Jane, and Jane's new address is '123 Other Street'." No ID is checked.</i>
WHAT AN ATTACKER COULD DO	Knowing only a customer's email (often guessable or scraped from social media), an attacker could redirect that customer's monthly pen-pal letters — and any other mail — to an address of their choosing. For a children's product, this is a particularly serious safety concern.
WHAT TO FIX	Send the customer a one-time confirmation link by email and only apply the address change after they click it. The same mechanism the password-reset flow already uses can be re-used.
EFFORT	1–2 developer days. The email-sending infrastructure already exists (Resend is wired up for password reset).

For the dev team: `artifacts/api-server/src/routes/enroll.ts` and `artifacts/api-server/src/routes/give-a-key.ts` (the `/give-a-key/po-box` handler). Re-use the `reset_token` pattern from `auth.ts` .

CRITICAL

3.4 Receipt upload accepts any file from anyone

Effort: Small

WHAT IT IS	When a Give-a-Key recipient uploads a photo of their PO-Box receipt, the server: (a) accepts any file type by trusting the filename's extension; (b) does not check the file size; (c) overwrites any earlier file with the same name; and (d) saves it to a <i>publicly readable</i> folder, meaning the resulting URL works for anyone on the internet who guesses it.
ANALOGY	<i>It's a donation drop-box on a public street that accepts anything you put in it, no size limit, and the contents can be viewed by anyone walking by.</i>
WHAT AN ATTACKER COULD DO	Upload very large files to run up your Supabase storage bill; upload disguised malware files to a URL that looks like it belongs to MailDay; or simply view other families' receipts (which may include names, addresses, and partial payment info).
WHAT TO FIX	Three small changes: (1) check the file's real type by reading its first few bytes, not just the filename; (2) reject anything over (say) 5 MB; (3) put the receipts in a <i>private</i> Supabase bucket and generate short-lived links when an admin needs to view one.
EFFORT	1 developer day plus a short Supabase storage settings change.

For the dev team: `artifacts/api-server/src/routes/give-a-key.ts` lines 166–187. Use a magic-byte sniffer (e.g. `file-type` npm package), cap `fileBuffer.length`, set the bucket to private, and replace `getPublicUrl` with `createSignedUrl` (60-min expiry).

CRITICAL

3.5 Influencer commissions can be inflated by anyone

Effort: Small

WHAT IT IS

There is a public URL that increments an influencer's "conversion" count and the commission they are owed. It accepts any affiliate code from any caller, and it also accepts a "parent ID" that tags a real customer to that influencer — again, with no proof.

ANALOGY

A tip jar that anyone can drop slips into, each slip reading "credit this barista \$14 for serving me", with no check that the customer was ever served — and the slips are tallied automatically against payroll.

WHAT AN ATTACKER COULD DO

Inflate a friendly influencer's payout, sabotage a competitor's commission records, or attach real customers to the wrong influencer to skew growth metrics.

WHAT TO FIX

Only the Shopify checkout (which we trust) should be allowed to record a conversion. Require an HMAC signature on this endpoint or move the conversion-recording logic entirely into the Shopify-orders webhook.

EFFORT

Half a day.

For the dev team: `artifacts/api-server/src/routes/influencers.ts` lines 250–271. Either add HMAC verification (shared secret with the calling Shopify script), or move the `conversions++` logic into `webhooks.ts`'s Shopify-order handler using the existing `X-Shopify-Hmac-Sha256` check, and delete the public route.

4. Important Issues — Fix This Month

These do not need to be fixed this week, but each is a real exposure that will start to bite as the business grows or as more people are given accounts.

IMPORTANT

4.1 Login key lasts 30 days and cannot be cancelled

Effort: Medium

PLAIN ENGLISH

Once you sign in, the key is valid for 30 days, and there is no "log everyone out" button. The current "Log out" button just deletes your local copy — it doesn't tell the server to invalidate the key. If you lose a laptop, that laptop can still log in for up to 30 days.

FIX

Shorter access keys (15 minutes), longer refresh keys (1–7 days), and a server-side "revoked tokens" list. Or move to a session library that already does this.

EFFORT

2–3 days.

IMPORTANT

4.2 Rate limiting only on the login screen

Effort: Small

PLAIN ENGLISH

You can only try to log in 20 times in 15 minutes per network address, which is good. But the "Forgot password" and "Change password" forms have no such limit, and the public forms (enroll, GAK apply, GAK PO-box) can be submitted as fast as someone can type. Anyone could send 10,000 password-reset requests in a minute to flood your email service.

FIX

Apply the same rate limit to all sensitive endpoints, plus add a small per-email cooldown (e.g., "you can only request a password reset once every 60 seconds for the same email").

EFFORT

Half a day.

IMPORTANT

4.3 Test endpoints and seed data live in production

Effort: Small

PLAIN ENGLISH

An admin user can press a button that creates five fake Give-a-Key applications and four fake donations. That button exists in the live system. It is unlikely an admin would press it by accident, but it is the kind of thing that should not be available outside a test environment.

FIX

Wrap the seed endpoint in an `if (NODE_ENV !== 'production')` check, or delete it.

EFFORT

1 hour.

IMPORTANT

4.4 Children's first names & interests are sent to an AI on every match

Effort: Medium

PLAIN ENGLISH

When the "Run Matching" button is pressed, the names, ages, interests, and home states of every unmatched child are sent to a third-party AI (Anthropic Claude) to suggest pairings. This is documented and works well, but the data leaving your building is not redacted — it leaves with real first names attached.

WHY IT MATTERS

For a children's product, the bar for data sharing is higher (COPPA, state child privacy laws). At minimum the families should know this happens; ideally the data sent to the AI is anonymised first.

FIX

Replace first names with internal codes ("Child A", "Child B", ...) before sending to the AI, and translate back on the way home. Also confirm Anthropic's data retention setting is set to "do not train on submitted data".

EFFORT

1 day plus a privacy-policy review.

IMPORTANT

4.5 The "Health" display says matched children are at risk

Effort: Small

PLAIN ENGLISH

The dashboard's "At risk" counter is overcounting. Children who are already happily matched are still flagged red because a hidden "days waiting for a match" clock keeps ticking after the match was made. This is purely a display bug, not a data bug — the matches themselves are fine. (See Section 6 for the visual proof.)

FIX

When a match is approved, stop the clock. Either clear the `match_guarantee_start_date` field or have the "health" calculation ignore children whose status is "Matched".

EFFORT

2–4 hours.

IMPORTANT

4.6 Server may not see real visitor IPs

Effort: Tiny

PLAIN ENGLISH

The login rate-limiter (20 tries per 15 minutes) is meant to be per visitor. But because the app sits behind Replit's network, the server may see Replit's address for every visitor instead of the visitor's true address. Result: either every visitor in the world counts as one (so 20 attempts kicks in instantly for everyone), or the limit is effectively bypassed.

FIX

Add one line to the server config: `app.set('trust proxy', 1)`. Then verify the right behaviour by attempting 21 fast logins.

EFFORT

15 minutes.

IMPORTANT

4.7 Onboarding links never expire

Effort: Small

PLAIN ENGLISH

When a new customer signs up via Shopify, we email them a personalised link that lets them add their child's details without a password. That link has no expiry date — once it has been emailed, it works forever, and lets anyone who has the URL add new children to that family's account.

FIX

Give onboarding links a 30-day expiry (matching most other "magic link" patterns), and rotate the token after a child has been added.

EFFORT

2–4 hours.

5. Useful Improvements — Next Quarter

None of these are blocking. They are the items that turn a working tool into a polished, professionally-run product.

ITEM	WHAT & WHY	EFFORT
Audit trail	Today, if an admin or VA changes something important — a child's safety flag, a parent's tier, a donation amount — there is no record of <i>who</i> changed it and <i>when</i> . An audit log is one extra database table and a few lines of code, and pays for itself the first time you need to ask "who did that?".	2–3 days
Two-factor authentication	Add a second factor (a 6-digit code from an app) for admin accounts. Industry baseline for tools that touch children's data.	2 days
Automated tests	Today there are no automated tests. As the codebase grows this means every change requires manual re-testing. A small core test suite (login, role boundaries, webhook signatures) would catch the most common breakages.	1 week initially, then ongoing
Dependency monitoring	The app uses ~100 third-party packages. Some of them publish security updates regularly. A free tool (e.g. GitHub Dependabot) can open a pull request when an update is available.	1 hour setup
Generic error responses	Some routes return raw database error messages to the user when something goes wrong. These messages can hint at the database structure. Catch them and return a generic "Something went wrong" with a reference ID.	1 day
Privacy policy refresh	The app shares subsets of data with Shopify, ReCharge, Klaviyo, Anthropic, Tremendous, Slack, and Resend. The privacy policy should name them and explain what each receives.	1 day (with legal)
Pagination for tables	Children, Parents, Matches, Cancellations, Influencers all currently load every row. At a few hundred records this slows down; at a few thousand it will break.	1–2 days total
HTML page metadata	The browser tab description still reads "MailDay Matching — built on Replit. Update this description to reflect the app." Minor, but it's visible whenever someone shares a link.	5 minutes
Accessibility audit	The component library is accessible by default, but a formal once-over (colour contrast, focus indicators, screen-reader walkthrough) will surface a handful of fixable items.	1 day

6. The "At Risk" Display Bug — A Worked Example

To make one of the findings concrete, here is the exact situation from the Children Directory screen you shared:

CHILD	STATUS	"DAYS WAITING"	SHOULD BE FLAGGED?
Marco	Unmatched	393 days	Yes — genuinely overdue
Zara	Paused	662 days	Yes — billing paused, needs attention
Kofi	Rematch Requested	378 days	Yes — waiting for a new pen pal
Amara	Matched	382 days	No — already matched
Noah	Matched	462 days	No — already matched
Lily	Matched	385 days	No — already matched

The dashboard says "At risk: 6". The honest answer is "At risk: 3" — Amara, Noah, and Lily are happily matched and should not be in that bucket. The fix is small and described in section 4.5.

WHY IT MATTERS Beyond accuracy, an inflated "at risk" number changes day-to-day behaviour. The team starts the day looking at a red number that is twice the real one, the Slack digest overstates urgency, and real problems can hide inside the noise.

7. Recommended Roadmap

If you would like to give a single instruction to the developer based on this report, the recommended order is:

Week 1 — "Stop the bleeding"

TASK	EST. EFFORT
Make webhook secrets mandatory (Section 3.2)	Half a day
Require email confirmation before changing addresses (Section 3.3)	1–2 days
Lock down receipt uploads + private storage (Section 3.4)	1 day
Remove or secure the public affiliate tracker (Section 3.5)	Half a day
Add <code>trust proxy</code> for correct rate limiting (Section 4.6)	15 minutes

Week 2 — "Tighten the locks"

TASK	EST. EFFORT
Move login key to a cookie + turn on CSP (Section 3.1)	1–2 days
Add short-lived tokens + server-side logout (Section 4.1)	2–3 days
Rate-limit forgot/reset/change password (Section 4.2)	Half a day
Remove the seed-data button from production (Section 4.3)	1 hour

Week 3 — "Clean the windows"

TASK	EST. EFFORT
Fix the "At Risk" health display (Section 4.5)	Half a day
Anonymise data sent to the matching AI (Section 4.4)	1 day
Expire onboarding links (Section 4.7)	Half a day
Update the page title/description (Section 5)	5 minutes

Quarter 2 — "Future-proof"

TASK	EST. EFFORT
Add an audit log table (Section 5)	2–3 days
Two-factor authentication for admins (Section 5)	2 days
Start a small automated test suite (Section 5)	1 week then ongoing
Dependency monitoring setup (Section 5)	1 hour
Accessibility & pagination polish (Section 5)	2–3 days
Privacy policy refresh with legal (Section 5)	1 day with counsel

Total estimate to clear the Critical + Important list: roughly **11–14 developer days**, ideally spread over three to four weeks. The Critical list alone (Section 3) is about **4 developer days**.

8. Glossary of Terms

Webhook	An automatic message one online service sends to another when something happens (e.g. Shopify telling MailDay "an order was just placed").
HMAC signature	A short code added to a webhook message that proves it really came from the claimed sender, the way a wax seal proves a letter wasn't tampered with.
Token	A short string of letters and numbers that says "the holder of this string is logged in as X". Like a paper wristband at a concert.
localStorage	A small storage area in the browser that any script on the page can read and write. Convenient but not private.
HttpOnly cookie	A special cookie that the browser will send to the server automatically, but scripts on the page <i>cannot</i> read. The recommended place to store login keys.
CSP (Content Security Policy)	A list of rules the server sends to the browser saying "only run scripts from these places". Blocks most cross-site-scripting attacks.
XSS (Cross-Site Scripting)	A bug where attacker-supplied text is rendered as code by the browser. The main way login keys in localStorage get stolen.
Rate limiting	Refusing to answer if requests come in too fast from the same place. A speed bump.
Bcrypt hash	A one-way scramble of a password. The original cannot be recovered. The standard way to store passwords.
Service-role key	A master key for the Supabase database that bypasses all access rules. The server has it; nothing else should.
RLS (Row Level Security)	A Supabase feature that locks down individual database rows by user. Currently off in MailDay because the Express server is the only thing allowed to touch the DB.
COPPA	A US law about collecting personal information from children under 13. Relevant because MailDay stores child data.
SPA (Single Page Application)	A website that loads once and then redraws itself as you click. MailDay is one.

Prepared for MailDay Leadership · May 19, 2026 · Confidential · Findings based on a review of the codebase, database schema, and live admin UI.